Ćwiczenie 2 – T-SQL (Zmienne, instrukcje sterujące, pętle)

Część teoretyczna

Zmienne w T-SQL

- Zmienne w T-SQL deklarujemy za pomocą instrukcji `DECLARE`.
- Do przypisywania wartości służy `SET` lub `SELECT`.
- Nazwy zmiennych muszą zaczynać się od znaku `@`.

Przykład:

```
DECLARE @liczba INT;
SET @liczba = 10;
```

Instrukcje sterujące

- `IF ... ELSE` – pozwala wykonać różne fragmenty kodu w zależności od warunku.
- Warunki logiczne mogą wykorzystywać operatory: =, <>, <, >, <=, >=.

Przykład:

```
IF @liczba > 0
    PRINT 'Liczba dodatnia';
ELSE
    PRINT 'Liczba ujemna lub równa zero';
```

Instrukcja CASE

- `CASE` pozwala podejmować decyzje na podstawie wielu warunków.
- Może być używana zarówno w zapytaniach `SELECT`, jak i w programach T-SQL.

Pętle

- W T-SQL dostępna jest pętla `WHILE`.
- Pętla wykonuje się tak długo, jak spełniony jest warunek logiczny.
- Wewnątrz pętli można modyfikować zmienne.

Instrukcje wypisywania wyników

- `PRINT` – wypisuje komunikaty tekstowe.
- Funkcja `CAST` lub `CONVERT` pozwala zamieniać liczby na tekst w komunikatach.

Operacje na danych

- `SELECT` – pobieranie danych do zmiennych.
- `UPDATE` – modyfikacja danych w tabelach.
- `ALTER TABLE` – zmiana struktury tabeli.

Część praktyczna

1. Napisz program, który dla zadanych dwóch liczb sprawdzi, która z nich jest większa i wypisze tekst: **„Wprowadzono dwie liczby: xxx i yyy. Większa liczba to xxx."** Jeżeli liczby są równe: **„Wprowadzono dwie liczby: xxx i yyy. Liczby są sobie równe."**

2. Napisz program, który dla numeru departamentu zdefiniowanego w zmiennej `@departmentID`:
   - wyświetli nazwę departamentu,
   - wyświetli liczbę pracowników w tym departamencie.

   Wynik przedstaw w postaci:
   - **Nazwa departamentu o identyfikatorze xxx to: xxxxx.**
     W departamencie pracuje xxxx pracowników.

3. Sprawdź, czy są pracownicy, którzy obchodzą dzisiaj urodziny. Jeżeli tak to wyświetl ich **imię** i **nazwisko**. Jeżeli nie to wypisz „**Nikt nie obchodzi dzisiaj urodzin**".

4. Napisz program wyliczający sumę pięciu kolejnych liczb naturalnych. Zliczanie rozpocznij od liczby zdefiniowanej w zmiennej `@liczbaStartowa`. Do rozwiązania użyj pętli `WHILE`.

5. Do tabeli Zoo.Zookepers dodaj atrybut Salary DECIMAL(8,2), który będzie zawierał wartość wynagrodzenia dla opiekuna zwierząt.
   Napisz program, który:
   - dla pracownika o wskazanym ID sprawdzi, czy jego wynagrodzenie jest **mniejsze od średniej**,
   - jeżeli tak – zwiększy wynagrodzenie o **10%**.

6. Zmodyfikuj program z zadania 5 tak, aby wynagrodzenie było zwiększane **tak długo**, aż będzie **większe od średniego wynagrodzenia**. Do rozwiązania użyj pętli.

7. Napisz program, który będzie wyliczał pole dla zadanej figury.

   A. Przygotuj dwie tabele:

   - RodzajFigury – zawierająca dwa atrybut RodzajFiguryID oraz NazwaFigury. Atrybut RodzajFiguryID powinien być kluczem głównym.

   - Figury – zawierającą dane różnych figur potrzebnych do policzenia pola. Tabela zawiera atrybuty:

     □ 1. FiguraID – klucz główny, wartość automatycznie numerowana

     □ 2. RodzajFiguryID – klucz obcy do tabeli RodzajFigury

     □ 3. Pozostałe pola potrzebne do wyliczenia pól poszczególnych fogur

   B. Wprowadź do tabeli `RodzajFigury` przykładowe nazwy: trójkąt, koło, kwadrat, prostokąt.

   C. Wprowadź do tabeli Figury dwie przykładowe figury dla każdego rodzaju

   D. Napisz program, który policzy pole dla zadanej wartości atrybutu `FiguraID`

8. Napisz program, który wypisze liczby od 1 do 100, ale pominie liczby podzielne przez 3.

9. Napisz program, który dla podanego wynagrodzenia pracownika, wypisze:

- „niskie wynagrodzenie” – poniżej 4000,
- „średnie wynagrodzenie” – 4000–7000,
- „wysokie wynagrodzenie” – powyżej 7000.